Quest Study — Guide Update

Implementation instructions for the IDE build: what to install, what to build, in what order

Standalone supplement for the IDE build. English.

About this update

This is a separate update document. It supplements the Quest Study UI Guide (Duolingo style) and the Quest Study Technical Specifications, and it contains only the updates settled after those documents were delivered.

Where this document differs from the UI Guide or the Technical Specifications, this document wins. One example is the boss-grade rule in Section 6, which replaces the earlier rule.

Read this document as build instructions, in order: install Section 0 first, follow the build order, then implement Sections 1–11. Every value given is the settled value — implement it exactly as written and take values from the constants in Section 0.5, never from literals scattered through the code.

0. Setup — install this first

0.1 Stack already in place — do not change it

Component	Use
Backend	Django + Django REST Framework, Python 3.11+
Frontend	React, responsive for phone and desktop browsers
Database	PostgreSQL 16+
AI model	Ollama, Qwen2.5 3B Instruct, local inference only
Host / dev OS	Windows 11 laptop until finalization

- **No second LLM.** Do not install another language model. PDF reading is parsing and OCR (Section 11); all generation uses the Qwen2.5 3B Instruct model already in the stack.
- **Server-side only.** Only Django calls Ollama. The browser and the React frontend never call Ollama directly.
- **No cloud or subscription AI.** Do not add a cloud parser, a cloud OCR service, or any paid AI API.

0.2 Python packages — add to requirements.txt

Package (pip name)	Purpose	Required
PyMuPDF	PDF text extraction, per page; also renders a page to an image for OCR	Yes — or pypdf, see note
pytesseract	Python wrapper that calls the Tesseract binary	Yes
Pillow	Image handling between page rendering and OCR	Yes

Note: *Install one PDF parser, not both: use PyMuPDF. Use pypdf only as a drop-in replacement for the extraction step if PyMuPDF cannot be installed; the rest of Section 11 is unchanged.*

```
pip install PyMuPDF pytesseract Pillow
```

0.3 System software — Tesseract is not a pip package

- **Install the Tesseract OCR binary separately.** On Windows 11, install the Tesseract program itself, not just the Python wrapper. Either add its install folder to PATH, or set the binary path in one settings value:

```
TESSERACT_CMD = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
```

- **Install two language data packs:** Indonesian (ind) and English (eng). School modules mix both; without ind, accuracy on Indonesian pages drops noticeably.
- **Select the language at call time:** pass lang="ind+eng" to every OCR call (Section 11).

Verify the installation before building Section 11:

```
tesseract --version
tesseract --list-langs   # output must list ind and eng
ollama list              # Qwen2.5 3B Instruct must be present
```

0.4 Optional — embedding (RAG) route only

Install nothing in this section for the default build. Keyword chunk selection (Section 11) needs none of it. Add these only if the embedding route is explicitly chosen for long modules:

Addition	Purpose
sentence-transformers with all-MiniLM-L6-v2, or nomic-embed-text via Ollama	Embed module chunks for similarity selection
pgvector extension in PostgreSQL 16+	Store chunk vectors in the existing database

```
-- only on the embedding route:
CREATE EXTENSION IF NOT EXISTS vector;
```

0.5 Constants — every number lives in one place

Create one backend constants location (a single settings or constants module) and define every value below there. Reference these names everywhere; do not hardcode the numbers in models, services, or frontend code. Changing a value later means editing this one place.

Constant	Value	Used in
DEFAULT_REGULAR_QUESTION_COUNT	5	§2
BOSS_QUESTION_COUNT	20 (fixed)	§2, §7
BOSS_TIMER_SECONDS	3600 (1 hour)	§7
BOSS_MAX_ATTEMPTS	3 total (2 retries)	§7

DAMAGE_PER_CORRECT	10	§6, §7
HP_PER_REQUIRED_CORRECT	10	§7
HP_PER_SPARE_QUESTION	10	§7
XP_PER_CORRECT	5	§6
XP_STREAK_BONUS	5	§6
STREAK_BONUS_FROM	3rd consecutive correct	§6
XP_WIN_BONUS	100	§6
XP_PER_HP_REMAINING	1 (wins only)	§6
TIER_DAMAGE	easy 5, normal 10, hard 20	§8
OCR_MIN_CHARS_PER_PAGE	50	§11
CHUNK_TOKENS	~500	§11
GROUNDING_TOKENS	2,000–4,000 per prompt	§11
GENERATION_RETRIES	1	§4

Build order

1. Install and verify Section 0 (packages, Tesseract, constants).
2. Add the model fields and migrations: Sections 1, 2, 8, 10, 11.
3. Build the scoring services and unit-test them against the worked examples: Sections 3, 5, 6, 7, 8.
4. Build the generation service (Section 4) and the PDF pipeline (Section 11).
5. Add the endpoints, permissions, and visibility filters: Sections 9 and 10.
6. Build the frontend pieces: question renderers, teacher review screen, boss attempt flow.
7. Build the Phase 2 question types (Section 3) only after Phase 1 passes.
8. Run the acceptance checklist at the end of this document.

1. Quests per subject

- **Add no cap.** Do not add a maximum-quests-per-subject setting, validation rule, or database constraint. Quest creation must never fail because a subject already has quests.
- **Keep the two creators.** Assignment Quests are created by the teacher; Revision Quests are created by the student. The count for a subject is simply however many each of them creates.
- **Rely on the existing filters.** Keep GET /api/quests/ filterable by type, subject, and status; the quest board organises by those filters.
- **Teacher count display.** Include a per-subject quest count in the teacher progress response (GET /api/progress/), computed as an aggregate count grouped by subject. Display only — no warning threshold and no blocking.

- **Revision-plan suggestion.** POST /api/ai/revision-plan/ may return a suggested split of a subject into revision quests by topic. Return it as a suggestion for the student to accept or edit; never create quests from it automatically.

2. Question count per quiz

Models

- **Add question_count (integer) to the Quiz model.** Set it when the quiz is created; the generator must produce exactly this many questions (Section 4).

Quiz type	question_count rule
Regular quest quiz	Teacher-settable; default DEFAULT_REGULAR_QUESTION_COUNT (5)
Boss exam	Fixed at BOSS_QUESTION_COUNT (20); not teacher-settable

API and validation

- **Accept question_count on quiz creation:** POST /api/quizzes/ takes topic, question_count, difficulty, and question types (Section 4).
- **Enforce the boss value server-side:** reject a boss quiz create or update whose question_count is not 20. Do not rely on the frontend to hold it at 20 — the count is tied to the HP derivation in Section 7.
- **Enforce the generated count:** after generation, count the returned questions. A result with any other count fails validation and follows the retry path in Section 4.

3. Question types

Data shape

Add a type field to each question in the stored quiz JSON (JSONB), with one payload shape per type:

type value	Payload to store
multiple_choice	stem, options list, correct option
true_false	stem, correct boolean — stored and rendered as a two-option multiple-choice question
fill_blank	stem with a blank, accepted_answers list of correct variants
ordering	items list, correct_order
matching	pairs list of left item and its match

Scoring — one scorer per type, one result: correct or not

- **multiple_choice:** correct when the selected option is the stored correct option.
- **true_false:** reuse the multiple-choice scorer and renderer; do not build a separate path.

- **fill_blank:** normalise before comparing — trim whitespace, ignore case — then accept any value in accepted_answers.
- **ordering:** correct only when the submitted order equals correct_order in full.
- **matching:** correct only when every submitted pair matches the stored pairs.
- **Boss rule for every type:** the scorer result is the whole result. A multi-part answer (ordering, matching) is a hit only if fully correct; there is no partial credit and no per-question timer.

Phasing

- **Build Phase 1 first:** multiple choice and true/false, including renderer, scorer, and generation prompt for each.
- **Build Phase 2 after Phase 1:** fill-in-the-blank, ordering, matching pairs. Each needs its own renderer, its own scorer, and its own generation prompt. Until all three exist for a type, quiz creation must not offer that type.
- **Do not implement:** open-ended or short-answer questions, drawing questions, and polls. Add no AI-judging path and no teacher-grading path for answers.

4. Auto-generation

Build a server-side generation service in Django. The teacher sets the shape — topic, question_count, difficulty, question types — and the service fills it in. Run these steps in order:

1. Look up the cache first: if a stored quiz already matches the topic and parameters, reuse it, including for a different class on the same subject. Generate once; never regenerate per student.
2. Build the prompt from the parameters, plus the grounded module excerpt when a module is attached (Section 11).
3. Call the local Ollama model (Qwen2.5 3B Instruct) from Django. The frontend never calls Ollama.
4. Parse the response as structured JSON and validate it: question count equals question_count, every question matches its type shape (Section 3), and every scored question has exactly one correct answer.
5. On any validation failure, retry generation exactly once (GENERATION_RETRIES = 1).
6. If the retry also fails, fall back to the template quiz bank. Never surface a raw model error to the teacher.
7. Store the result as a draft quiz (Section 10) with its generation parameters, so it can be reviewed, regenerated, and reused.

- **Endpoint:** POST /api/ai/generate-quiz/ triggers this service and returns the created draft quiz. POST /api/ai/revision-plan/ is unchanged.
- **Quests are not AI-generated here.** The teacher authors Assignment Quests and students author their own Revision Quests. This service generates quiz questions and revision plans only. The single exception is the module draft quest split in Section 9, which is also a draft for teacher approval.

5. Grading

- **Store one integer score per attempt.** Add score (integer, 0–100) to the quiz attempt, plus the counts it came from. Return scores as integers in every API response and screen; show no decimals.
- **Regular quiz logic:** score = correct answers ÷ total questions × 100, rounded to the nearest whole number. Nothing else enters a regular quiz score.

- **Keep XP separate.** XP and levels are the game reward system, recorded through the existing central XP function. Do not derive a regular grade from XP. The only XP-to-score conversion in the app is the boss formula in Section 6.
- **Auto-score everything.** Every score comes from the Section 3 scorers. Add no teacher-grading field, queue, or screen.
- **Do not add letter grades.** Add no letter field, no conversion table, and no letter in any response or screen. Scores are 0–100 only.
- **Best attempt counts.** For each student and quiz, the score of record is the highest attempt score. A later, lower attempt never replaces it.
- **Teacher view:** GET /api/progress/ returns, per student per quiz: the attempts, the best score, and the percentage. The best score and the percentage are the same integer.

6. Boss score formula

Implement the boss score as a server-side scoring function. It replaces any earlier boss grade based on correct answers ÷ questions answered — remove that rule wherever it exists. The regular quiz keeps the plain percentage in Section 5.

```
score = round_half_up(xp_earned / xp_max * 100)
# round half up to the nearest whole number;
# do not use Python round() (banker's rounding)
```

XP earned — compute in answer order

Component	Rule
Correct answer	+XP_PER_CORRECT (5) per correct answer
Streak bonus	+XP_STREAK_BONUS (5) per correct answer from the 3rd consecutive correct answer onward
Win bonus	+XP_WIN_BONUS (100), only on defeating the boss
HP bonus	+XP_PER_HP_REMAINING (1) per HP remaining, only on a win

- **Track the streak while scoring:** keep a consecutive-correct counter; increment it on a correct answer, reset it to zero on a wrong answer. Award the streak bonus on a correct answer when the counter reaches STREAK_BONUS_FROM (3) or more.
- **On a loss, award neither bonus:** no win bonus and no HP bonus. Per-answer and streak XP already earned still count.

XP max — compute from the fight setup, never hardcode it

- **Formula:** XP max = required correct × XP_PER_CORRECT + (required correct − 2) × XP_STREAK_BONUS + XP_WIN_BONUS + maximum player HP × XP_PER_HP_REMAINING.
- **Standard setup check:** 50 (10 × 5) + 40 (8 × 5) + 100 + 100 = 290.

Store the breakdown on the attempt

Persist xp_per_answer, xp_streak, xp_win, xp_hp, xp_total, xp_max, and the final integer score on the boss attempt. The frontend displays the returned values; it never computes the official score, XP, or outcome itself.

Worked examples — use as unit-test fixtures (standard setup)

Result	XP earned	Score
Perfect: 10 right, 0 wrong, HP 100	290	100
10 right, 1 wrong, HP 100	280	97
10 right, 2 wrong in a row, HP 90	270	93
10 right, 10 wrong alternating, HP 100	250	86
Loss after 8 right	70	24

7. Boss scaling and the boss attempt

Derive HP at quiz setup — store the derived values

Value	Rule
Boss HP	Required correct answers × HP_PER_REQUIRED_CORRECT (10)
Player HP	Spare questions × HP_PER_SPARE_QUESTION (10)
Spare questions	Total questions − required correct answers
Damage per correct answer	DAMAGE_PER_CORRECT (10), fixed

- **Standard setup:** 20 total, 10 required → boss HP 100, spare 10, player HP 100.
- **Teacher dials:** total question count, required correct answers, and the content difficulty of the questions. Derive both HP values from the dials with the table above; compute XP max from the same values (Section 6), so the score scales with the setup automatically.
- **Validate every setup:** reject required correct greater than total questions, and reject any boss HP or player HP that does not equal its derived value.

Worked example — medium setup, use as a test fixture: 70 total, 40 required → boss HP 400, spare 30, player HP 300, perfect-run XP max 790 (200 + 190 + 100 + 300).

Guardrail: Do not build this combination from the standard rules: 10 total questions with 400 boss HP and 100 player HP. At 10 damage per correct answer, 400 boss HP needs 40 correct answers from 10 questions, and 10 total with 40 required leaves no spare questions to derive player HP from. The setup validation above must reject it; it could only exist under a separate perfect-run or HP-floor decision, which has not been made.

Attempt state — store on the attempt

- boss_hp_max, boss_hp_current, player_hp_max, player_hp_current.
- consecutive_wrong counter and consecutive-correct streak counter.

- started_at and deadline = started_at + BOSS_TIMER_SECONDS (3600).
- attempt_number for the student and quiz (1, 2, or 3).
- The question order and option order fixed for this attempt (below).

Endpoints

- **Start attempt:** POST /api/quizzes/{id}/attempts/ creates the attempt. On the server, at attempt start: shuffle the question order and shuffle each question's answer options, store both orders on the attempt, and return them in that order. The response carries: the questions in attempt order with shuffled options and no correct-answer markers, boss_hp_current and boss_hp_max, player_hp_current and player_hp_max, the deadline, and the attempt number with attempts remaining.
- **Check an answer:** submit each answer to the server for checking; the server applies the Section 3 scorer, updates HP, streaks, and XP, and returns the updated boss and player HP, the streak, and the XP earned so far. A question is answered once per attempt; never accept a second answer to it.
- **Read attempt:** GET /api/quizzes/{id}/attempts/{attempt_id}/ returns the stored attempt state, and on a finished attempt the outcome, the Section 6 XP breakdown, and the integer score.

Fight logic — apply in this order per answer

1. If the current time is past the deadline, finish the attempt as a fail (time expired) and accept no further answers.
2. Score the answer with its Section 3 scorer.
3. Correct: boss_hp_current −= DAMAGE_PER_CORRECT (10), floor at 0; update streak and XP (Section 6). If boss HP reaches 0, finish as a win.
4. Wrong: apply the tier damage rule in Section 8. If player HP reaches 0, finish as a fail.
5. If all questions are answered and the boss still has HP, finish as a fail (questions ran out).

- **Timer is a fail cap only:** time never enters the score. Finishing faster earns nothing.
- **Retries:** enforce BOSS_MAX_ATTEMPTS server-side — at most 2 retries after a loss, 3 attempts total per student per boss quiz. Reject a fourth attempt start.
- **Display:** return and show the numeric 0–100 score only. Do not add a rank field, rank calculation, or rank display.

8. Difficulty tier

Models and API

- **Add difficulty_tier to the boss Quiz model:** choices easy, normal, hard. Set it at boss quiz creation and return it in the quiz detail and in the attempt state.
- **Scope:** the tier controls only the damage the player takes per wrong answer. It is independent of the question count and of the content difficulty used at generation. Do not let it change the damage the player deals, which stays at DAMAGE_PER_CORRECT (10).

Tier	Damage taken per wrong answer
Easy	5 HP
Normal	10 HP

Hard	20 HP

Logic — wrong-answer streak rule

1. Keep a consecutive_wrong counter on the attempt; reset it to zero on any correct answer.
2. On a wrong answer, if the counter is 0 (first wrong answer in the streak), apply no damage.
3. On each further consecutive wrong answer, apply player_hp_current −= TIER_DAMAGE[tier], floor at 0, then apply the Section 7 fail check.

Note: *Read the values from the TIER_DAMAGE constant (Section 0.5). Sanity check on Normal: 11 consecutive wrong answers lose the fight — the first is free and the next 10 cost 10 HP each.*

9. Teacher workflow

Implement the workflow as these six steps, in this order. Difficulty is set before generation because the difficulty settings are the generation inputs.

Step	Build
1. Create quest	POST /api/quests/ (teacher) or POST /api/quizzes/ for a boss exam. New items start as drafts (Section 10).
2. Upload module PDF	Attach a module with the Section 11 upload endpoint.
3. Set difficulty	Save the difficulty and tier; auto-set the amounts of quests and questions from them (Sections 2 and 7). Accept teacher overrides of the amounts through the same update, subject to the Section 7 validation.
4. Generate	POST /api/ai/generate-quiz/ (Section 4). From a module, also draft a proposed quest split — for example one quest per sub-topic in the module — as drafts for the teacher to edit.
5. Review and publish	Teacher edits, regenerates, or publishes on the review screen (Section 10).
6. Students see published quests	Student lists and details return published items only. Questions are returned only inside a quiz attempt or boss attempt, never before it.

- **Boss variant:** run the same six steps with generation and setup restricted to the boss-fight aspects: the boss question set, the Section 7 HP setup, the Section 8 tier, and the Section 7 timer.
- **Teacher-lite boundary:** the module upload is teacher source material, not student work. Do not add student file submission or file grading anywhere in this workflow.
- **Approval gate:** no AI-generated quest, question, or quest split reaches a student without teacher publish (Section 10).

10. Review before publish

Models

- **Add status to both the Quest and the Quiz models:** choices draft and published. Every AI-generated quest, quiz, and boss exam is created with status draft.

Endpoints and actions

- **Edit:** PATCH /api/quests/{id}/ and PATCH /api/quizzes/{id}/ update a draft; it stays a draft after editing.
- **Regenerate:** add POST /api/quizzes/{id}/regenerate/ for quizzes and AI-drafted quests. It re-runs generation (Section 4) with the stored parameters, replaces the draft content, and leaves the status as draft.
- **Publish:** add POST /api/quests/{id}/publish/ and POST /api/quizzes/{id}/publish/. Each sets status to published in one transition. Publishing is teacher-only.
- **Review screen data:** the teacher review screen lists the teacher's drafts and, for each draft, exposes exactly these three actions: edit, regenerate, publish. Include the module's OCR page flags (Section 11) in the draft payload so the screen can show them.

Visibility rules — enforce on the server

- Filter every student-facing quest and quiz list and detail query to status published. A direct student request for a draft returns not found.
- Return a quest's questions only inside a quiz attempt, and a boss quiz's questions only inside a boss attempt (Section 7). Never include questions in a quest list or detail response.
- Drafts are visible to teachers only.

11. PDF module pipeline

Do not install or call a second AI for reading. Extraction is parsing (PyMuPDF) and OCR (Tesseract); understanding the extracted text is done by the same Ollama Qwen2.5 3B Instruct model used in Section 4.

Models

Add a module model linked to its quest (or subject and topic) and to the uploading teacher, with these fields:

- **file:** the uploaded PDF, stored on the server (FileField).
- **extracted_text:** the full extracted text, in page order, stored in the database.
- **page_count:** number of pages in the PDF.
- **ocr_pages:** a list of the page numbers read by OCR (Section 11, pipeline step 4).
- **source:** upload or paste, recording how extracted_text was supplied.

Endpoint

- **Upload:** add POST /api/modules/ accepting multipart form data with the PDF file and its quest, subject, or topic. Teacher-only. Run the pipeline below on upload and return the module record, including ocr_pages.
- **Paste fallback:** accept pasted module text on the same resource instead of a file: store it directly as extracted_text with source paste and skip pipeline steps 2–4. The rest of the pipeline is unchanged.

Pipeline — run in order on every upload

1. Save the file and create the module record.

2. Parser pass: with PyMuPDF, extract each page's text layer, page by page.
3. Decide per page, not per document: a page whose extracted text, stripped, is shorter than OCR_MIN_CHARS_PER_PAGE (50) goes to OCR. All other pages keep their parser text.
4. OCR pass, flagged pages only: render the page to an image with PyMuPDF, run it through pytesseract with lang="ind+eng" (binary and language data from Section 0.3), and record the page number in ocr_pages.
5. Assemble extracted_text from the parser and OCR results in page order, so the module reads as one continuous text, and save it.
6. Chunk the text into chunks of about CHUNK_TOKENS (~500) tokens.
7. Select the chunks relevant to the topic by keyword matching against the topic terms. This is the default and needs no embedding install. On the optional embedding route (Section 0.4), select by vector similarity instead and store the vectors with pgvector.
8. Build the grounded excerpt from the selected chunks, between 2,000 and 4,000 tokens (GROUNDING_TOKENS), and pass it into the Section 4 generation prompt as grounding. Generation, validation, caching, and review then follow Sections 4 and 10 unchanged.

Review flag

- Return ocr_pages with the module, and carry it into the teacher review payload (Section 10).
- On the review screen, flag the OCR pages to the teacher — for example: “Pages 4–5 were read from scans — check the questions against the module.” OCR misreads formulas, handwriting, and diagram labels most often, so those pages are where the teacher checks twice.

Acceptance checklist

Verify each item before handing the build back. Each check must pass exactly as written.

- **[ ] Setup:** PyMuPDF, pytesseract, and Pillow are installed; tesseract --list-langs lists ind and eng; ollama list shows the Qwen2.5 3B Instruct model; every Section 0.5 value is read from the constants location, with no matching literal elsewhere.
- **[ ] 1. Quests per subject:** creating many quests in one subject never fails; the teacher response shows the per-subject count; a revision-plan suggestion creates nothing until the student accepts it.
- **[ ] 2. Question count:** a regular quiz defaults to 5 and accepts a teacher-set count; the generator's result is rejected and retried when its count differs; a boss quiz cannot be created or updated with a count other than 20.
- **[ ] 3. Question types:** Phase 1 multiple choice and true/false render, score, and generate; fill-blank accepts a listed variant with different case and surrounding spaces; ordering and matching score correct only when fully correct; no open-ended, drawing, or poll type exists anywhere.
- **[ ] 4. Auto-generation:** a second request with the same topic and parameters reuses the cached quiz; invalid JSON is retried exactly once, then the template bank is used; the result is a draft; the frontend makes no Ollama call.
- **[ ] 5. Grading:** a regular attempt stores and returns an integer 0–100 score with no decimals and no letter anywhere; the teacher view shows attempts, best score, and percentage per student per quiz, and a later lower attempt does not change the best score.
- **[ ] 6. Boss score:** all five worked examples score exactly 100, 97, 93, 86, and 24 in unit tests; a loss stores no win bonus and no HP bonus; XP max is computed, not hardcoded.

- **[ ] 7. Boss scaling and attempt:** the medium setup fixture derives boss HP 400, player HP 300, and XP max 790; setup validation rejects 10 total with 400 boss HP and 100 player HP; two attempts of the same quiz return different question and option orders; the start response carries both HP values and the deadline; all three fails work — HP reaches zero, deadline passes, questions run out with the boss alive; a fourth attempt start is rejected.
- **[ ] 8. Difficulty tier:** on Easy, Normal, and Hard, the second consecutive wrong answer costs exactly 5, 10, and 20 HP; the first consecutive wrong answer costs 0 on every tier; a correct answer resets the streak; on Normal, 11 consecutive wrong answers lose the fight.
- **[ ] 9. Teacher workflow:** the six steps complete in order end to end for a quest and for a boss exam; difficulty saved before generation is what generation receives; students cannot retrieve questions before starting an attempt.
- **[ ] 10. Review before publish:** AI-generated items start as drafts and are absent from student lists; a direct student request for a draft returns not found; edit keeps a draft a draft; regenerate replaces the content and keeps it a draft; publish makes exactly that item visible to students.
- **[ ] 11. PDF pipeline:** a text PDF is extracted with no OCR and an empty ocr_pages; a mixed module sends only its low-text pages to OCR, lists them in ocr_pages, and reads in page order; the review screen flags those pages; generation receives a grounded excerpt of 2,000–4,000 tokens; a pasted-text module generates questions with no file involved.